Assignment 6

Tree-based approaches

Author: Samuel Fredric Berg

Student ID: sb224sc

Date: 2026-01-26

Course: Machine Learning 4DT905

Conceptual

1. **Bagging** focuses on reducing the variance. Trains several models in parallel and independently on different subsets of the data. These trees are deep, complex and avraging them to "smooth out" the overfit. Meanwhile **Boosting** focuses on reducing bias. Trains models sequentially, where each model tries to correct the errors of the previous one. Also uses shallow trees and stack them to learn complex patterns with time. By creating a forest of slightly different trees cancels out the noise in the data. Using **Bootstrapping** to ensure diversity among the trees.

2.

- **Random Forests**:
  - Parallel training
  - Equal weighting
  - Every tree get one equal vote
  - Very robust to overfitting
- **AdaBoost**:
  - Sequential training
  - Dynamic weighting, higherweight prioritized next iteration
  - More accurate tree the more influence
  - Sensitive to noisy data and outliers

Practical

Imports

In [3]:
```
import pandas as pd
import seaborn as sns
```

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import confusion_matrix, mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
```

Load Data

In [4]:
```
df = pd.read_csv("../data/Carseats.csv", index_col=0)
```

Number of features and names

In [5]:
```
df_names = df.columns.tolist()
print(f"Number of columns: {len(df_names)}")
print(f"Column names: {df_names}")
```

```
Number of columns: 11
Column names: ['Sales', 'CompPrice', 'Income', 'Advertising', 'Population', 'Price',
'ShelveLoc', 'Age', 'Education', 'Urban', 'US']
```

Statistatical summary of features

In [6]:
```
print(df.describe())
print(df["ShelveLoc"].value_counts())
print(df["Urban"].value_counts())
print(df["US"].value_counts())
```

```
            Sales   CompPrice      Income  Advertising  Population  \
count  400.000000  400.000000  400.000000   400.000000  400.000000
mean     7.496325  124.975000   68.657500     6.635000  264.840000
std      2.824115   15.334512   27.986037     6.650364  147.376436
min      0.000000   77.000000   21.000000     0.000000   10.000000
25%      5.390000  115.000000   42.750000     0.000000  139.000000
50%      7.490000  125.000000   69.000000     5.000000  272.000000
75%      9.320000  135.000000   91.000000    12.000000  398.500000
max     16.270000  175.000000  120.000000    29.000000  509.000000

            Price         Age   Education
count  400.000000  400.000000  400.000000
mean   115.795000   53.322500   13.900000
std     23.676664   16.200297    2.620528
min     24.000000   25.000000   10.000000
25%    100.000000   39.750000   12.000000
50%    117.000000   54.500000   14.000000
75%    131.000000   66.000000   16.000000
max    191.000000   80.000000   18.000000
ShelveLoc
Medium    219
Bad        96
Good       85
Name: count, dtype: int64
Urban
Yes    282
No     118
Name: count, dtype: int64
US
Yes       257
No        142
Yes         1
Name: count, dtype: int64
```

Number of datapoints

In [7]:
```
print(f"Number of datapoints: {len(df)}")
```

```
Number of datapoints: 400
```

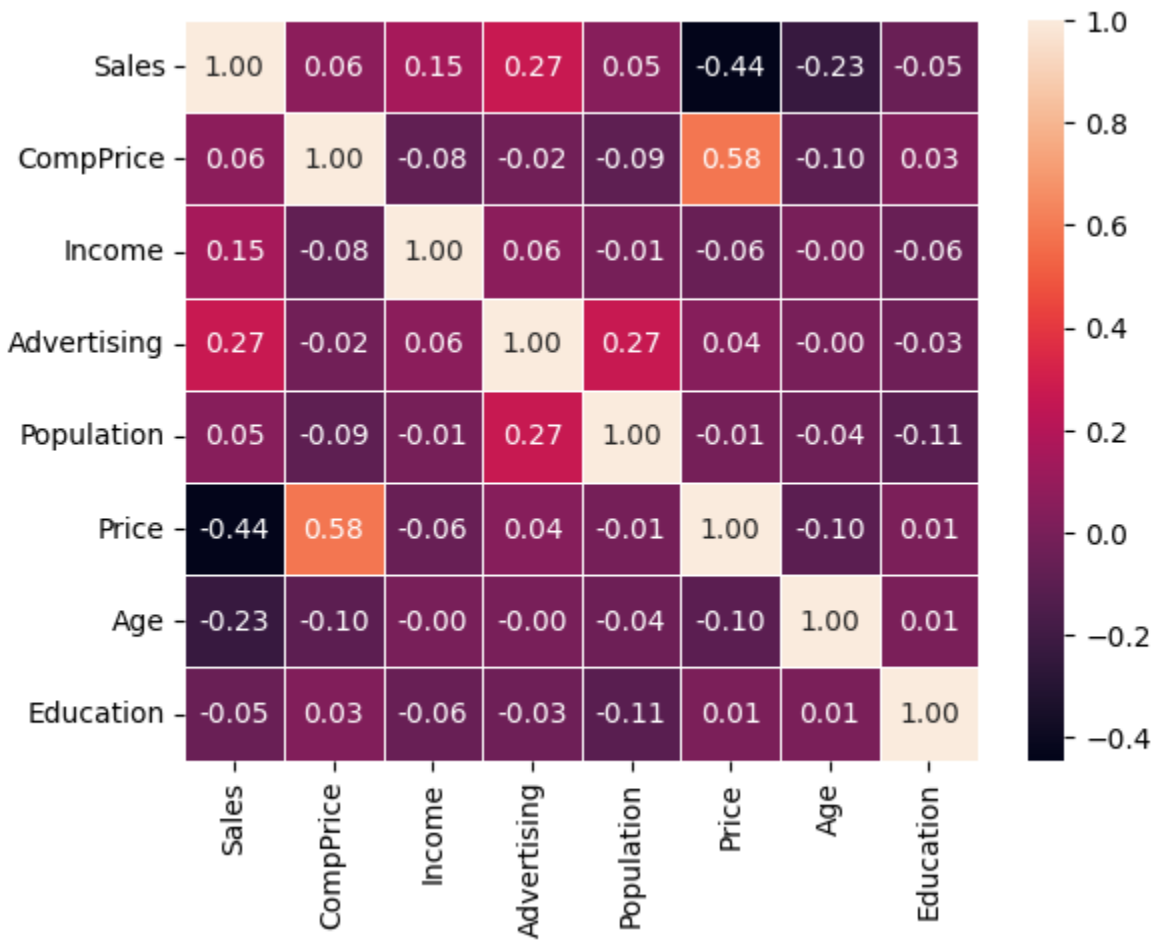
Display data in table format

In [8]:
```
print(df.head(5))
```

```
   Sales  CompPrice  Income  Advertising  Population  Price ShelveLoc  Age  \
1   9.50        138      73           11         276    120       Bad   42
2  11.22        111      48           16         260     83      Good   65
3  10.06        113      35           10         269     80    Medium   59
4   7.40        117     100            4         466     97    Medium   55
5   4.15        141      64            3         340    128       Bad   38

   Education Urban      US
1         17   Yes     Yes
2         10   Yes     Yes
3         12   Yes  Yes
4         14   Yes     Yes
5         13   Yes      No
```

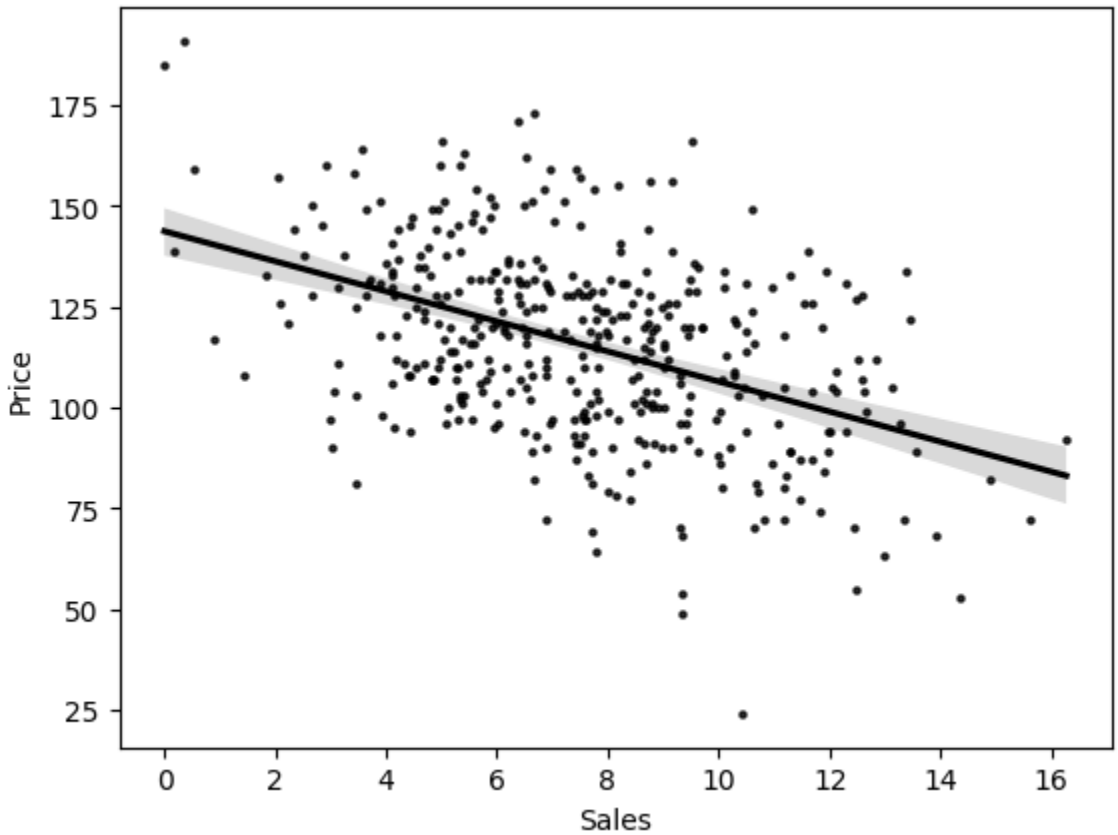
Correlation matrix

In [9]:
```
sns.heatmap(
    df.drop(columns=["ShelveLoc", "Urban", "US"]).corr(),
    annot=True,
    fmt=".2f",
    linewidths=0.5,
)
plt.show()
```



Scatter plot

In [10]:
```
sns.regplot(
    x="Sales",
    y="Price",
    data=df,
    scatter_kws={"s": 5, "color": "black"},
    line_kws={"color": "black"},
)
plt.show()
```



Interpretation

We are looking at "Sales" as our response variable and the most correlated feature is "Price" ($r \approx -0.44$). The correlation is negative, meaning that as the price increases, the sales decrease, and vice versa. The second most correlated feature is "Advertising" ($r \approx 0.27$), which has a positive correlation with sales, as advertising spend increases, sales also increase.

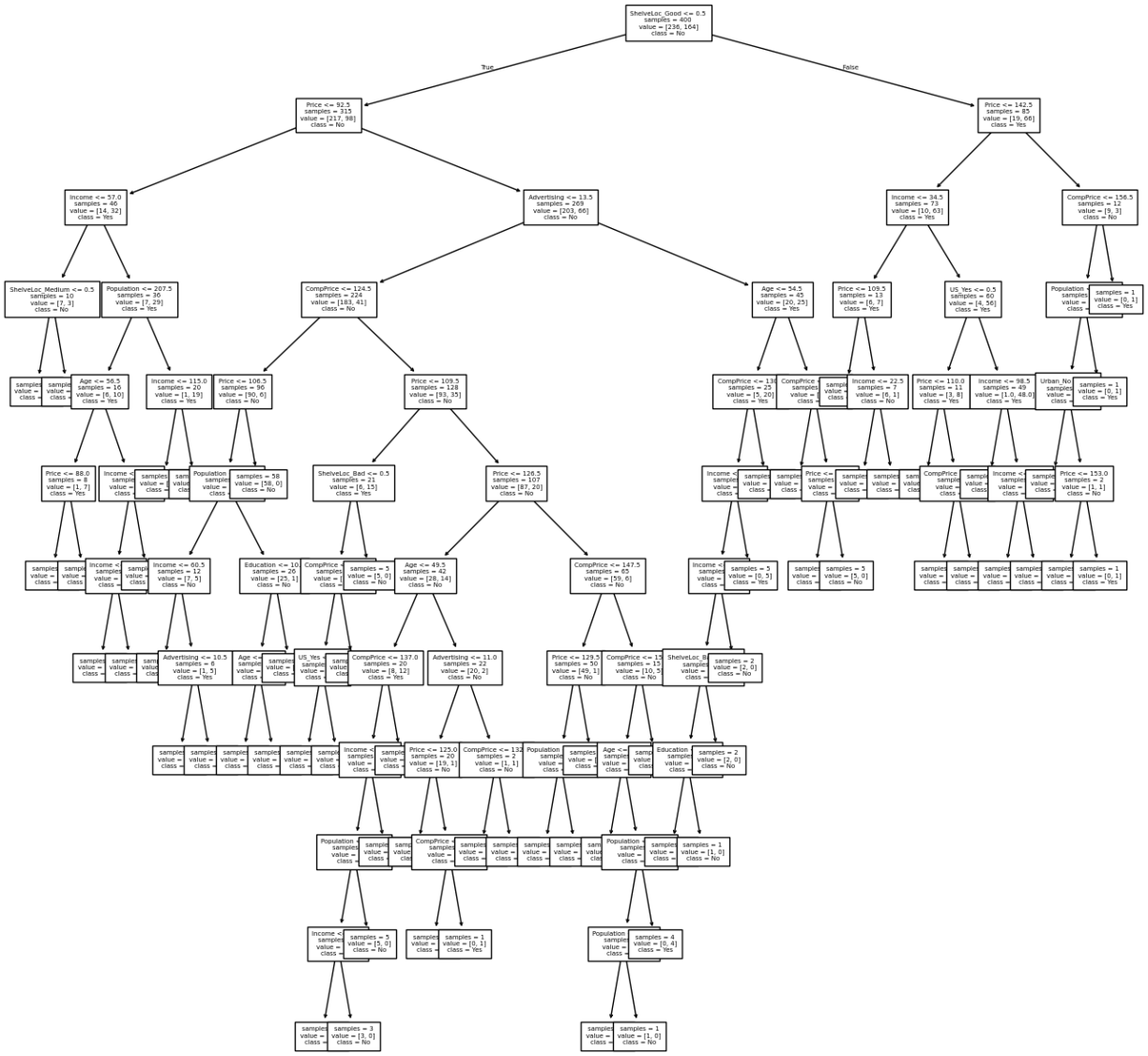
Regarding multicollinearity among predictors: "CompPrice" and "Price" show a moderate positive correlation ($r \approx 0.58$), suggesting that competitor pricing and shelf pricing tend to move together. "Age" and "Advertising" are weakly negatively correlated. "Population" and "Education" are nearly uncorrelated with both the target and with other predictors.

The strongest multicollinear pair is "CompPrice" and "Price". When both are included in a model together, their shared variance may reduce the individual significance of each predictor. The least correlated features with "Sales" are "Population" and "Education", indicating they are likely to be the least useful predictors.

In [11]:
```
df_encoded = pd.get_dummies(df, columns=["ShelveLoc", "Urban", "US"])
df["High"] = ["No" if sales <= 8 else "Yes" for sales in df["Sales"]]

X = df_encoded.drop(columns=["Sales"])
Y = df["High"]

tree = DecisionTreeClassifier()
tree.fit(X, Y)

print(f"Number of leaves: {tree.get_n_leaves()}")
```

```
Number of leaves: 61
```

In [20]:
```
plt.figure(figsize=(16, 16))
plot_tree(
    tree,
    feature_names=X.columns,
    class_names=["No", "Yes"],
    impurity=False,
    fontsize=5,
)
plt.show()
```



Interpretation

Each node in the tree represents a feature and a threshold value, if the threshold is met the left branch is followed, otherwise the right branch. When it raches the bottom of the tree it returns an answer (if "High" is Yes or No).

In [21]:
```
df_encoded = pd.get_dummies(df, columns=["ShelveLoc", "Urban", "US"], drop_first=Tr
X = df_encoded.drop(columns=["Sales", "High"])
Y = df["High"]

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.5, random_sta

tree_model = DecisionTreeClassifier(random_state=1)
tree_model.fit(X_train, Y_train)
Y_pred = tree_model.predict(X_test)

conf_matrix = confusion_matrix(Y_test, Y_pred)

print(conf_matrix)
```

```
[[94 25]
 [30 51]]
```

Interpretation

$$\begin{bmatrix} \text{True Negative (TN)} & \text{False Positive (FP)} \\ \text{False Negative (FN)} & \text{True Positive (TP)} \end{bmatrix} = \begin{bmatrix} 94 & 25 \\ 30 & 51 \end{bmatrix}$$

$$Precision = \frac{TP}{TP + FP} = \frac{51}{51 + 25} \approx 0.6710$$

$$Recall = \frac{TP}{TP + FN} = \frac{51}{51 + 30} \approx 0.6296$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{51 + 94}{51 + 94 + 25 + 30} \approx 0.7250$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = 2 \cdot \frac{0.6710 \cdot 0.6296}{0.6710 + 0.6296} \approx 0.6496$$

This indicates a precision of 67.10%, a recall of 62.96%, an accuracy of 72.50% and this gives us an F1-score of 64.96%.

Looking at the confusion matrix, the model shows a tend to misclassify actual "Yes" observations as "No", there are 30 false negatives compared to 25 false positives. This means the model is slightly biased toward predicting low sales ("No"), which leads to a higher false negative rate (37%) than false positive rate (21%).

In [26]:
```
path = tree_model.cost_complexity_pruning_path(X_train, Y_train)

cv_scores = []
best_accuracy = 0.0
best_alpha = 0.0
for alpha in path.ccp_alphas:
    pruned_tree = DecisionTreeClassifier(random_state=0, ccp_alpha=alpha)
    scores = cross_val_score(pruned_tree, X_train, Y_train, cv=5)
    accuracy = scores.mean()
    cv_scores.append(accuracy)
    if accuracy > best_accuracy:
        best_accuracy = accuracy
```

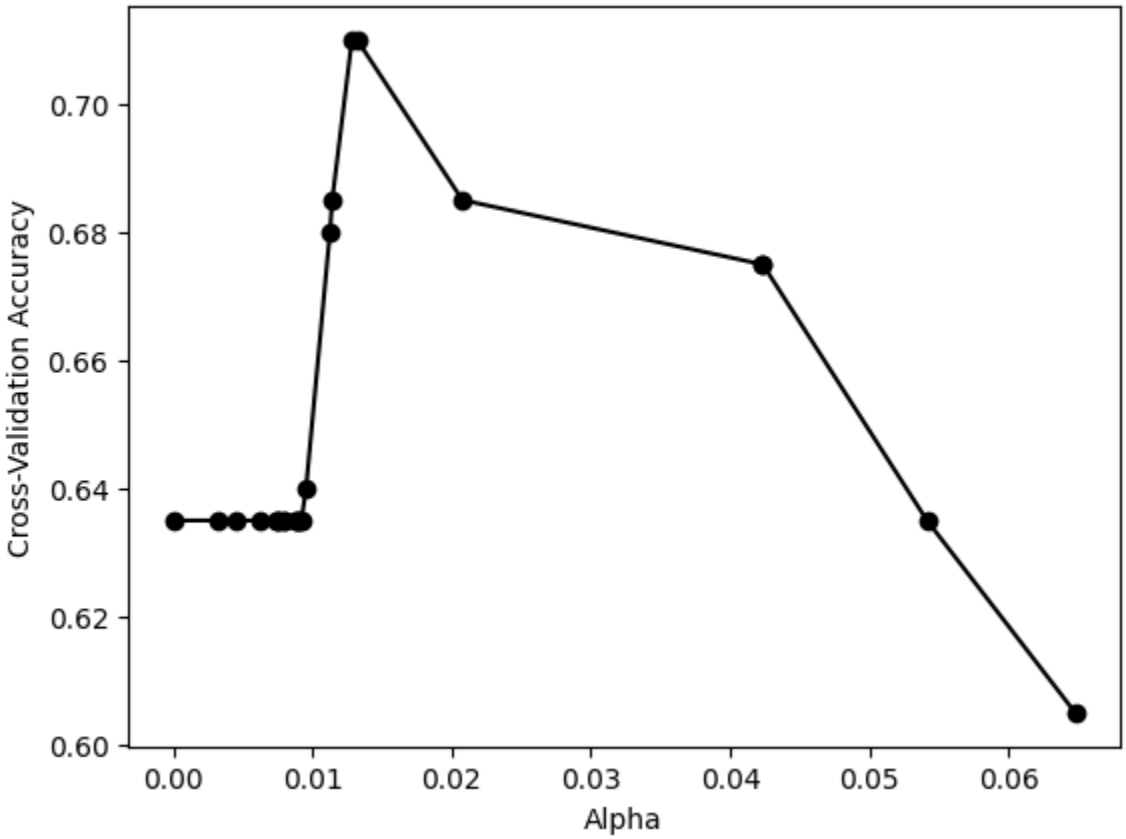
```
        best_alpha = alpha

print(f"Best accuracy: {best_accuracy:.4f}, with alpha: {best_alpha:.4f}")
```

```
Best accuracy: 0.7100, with alpha: 0.0128
```

Interpretation

Pruning reduces the accuracy form 72.5% to 71.00%. Meaning the pruned tree is less accurate than the unpruned tree.

In [27]:
```
plt.plot(path.ccp_alphas, cv_scores, marker="o", color="black")
plt.xlabel("Alpha")
plt.ylabel("Cross-Validation Accuracy")
plt.show()
```

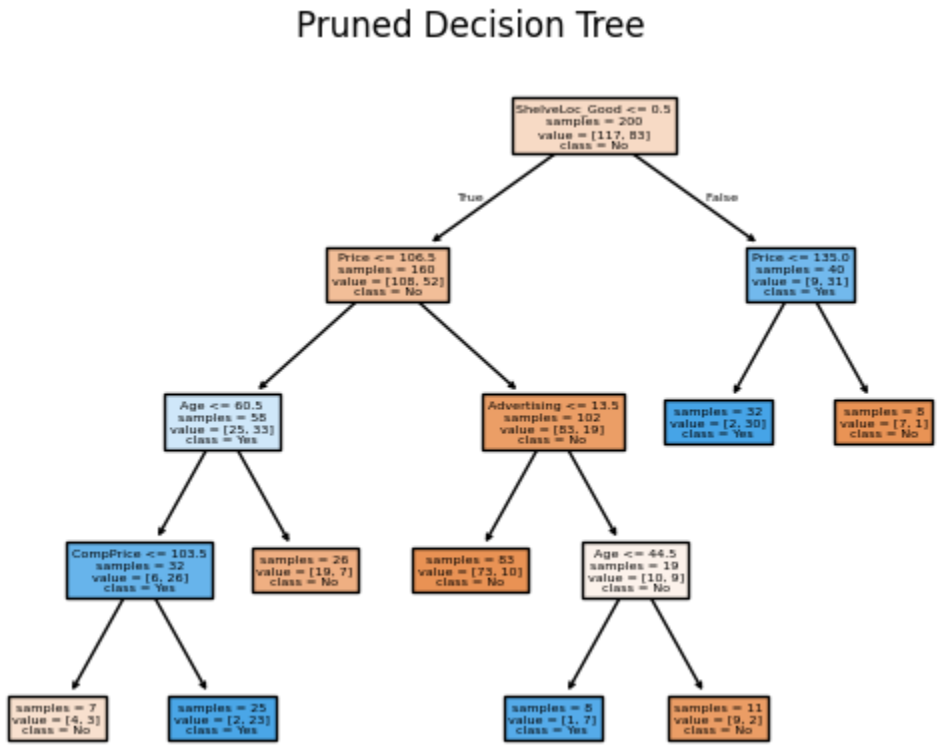


Interpretation

Previous result in graph form which indicates that at alpha 0.0128 provides us with an accuracy of 71.00%.

In [28]:
```
pruned_tree = DecisionTreeClassifier(random_state=0, ccp_alpha=best_alpha)
pruned_tree.fit(X_train, Y_train)

plot_tree(
    pruned_tree,
    filled=True,
    feature_names=X_train.columns,
    class_names=["No", "Yes"],
```

```
    impurity=False,
)
plt.title("Pruned Decision Tree")
plt.show()
```



In [29]:
```
Y_pred = pruned_tree.predict(X_test)

conf_matrix = confusion_matrix(Y_test, Y_pred)
print(conf_matrix)
```

```
[[100  19]
 [ 36  45]]
```

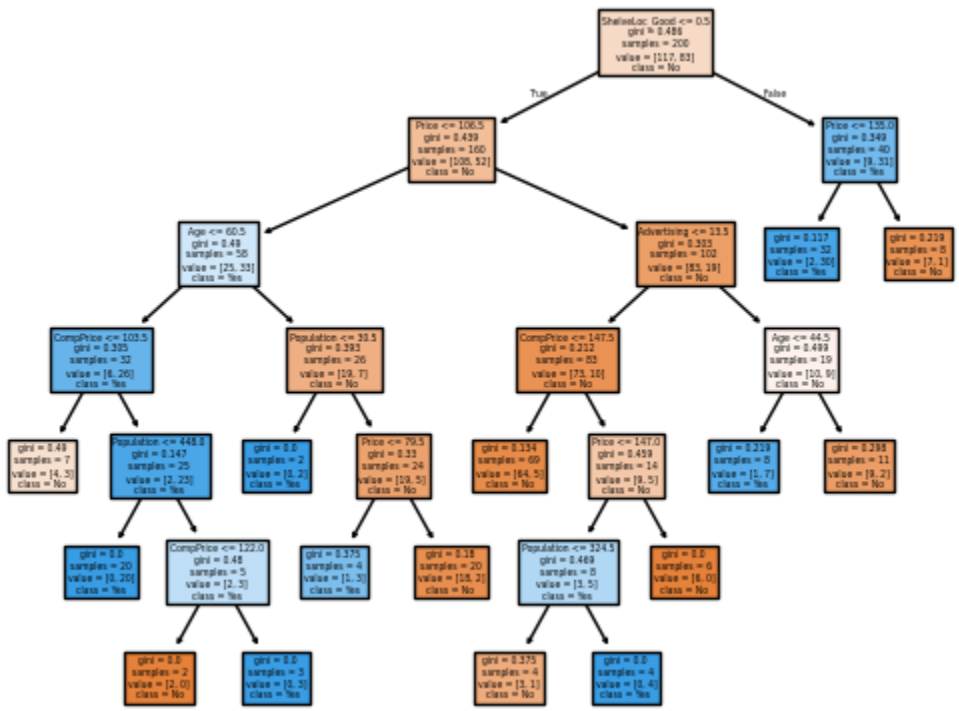
Interpretation

The pruned tree (selected by minimising cross-validation error) achieves the same test accuracy of 72.50% as the unpruned tree, while producing a considerably simpler and more interpretable tree structure. Examining the confusion matrix confirms the same overall pattern: the model still has more false negatives than false positives, indicating a slight bias toward predicting "No". The benefit of pruning here is not a gain in accuracy, but a reduction in model complexity that lowers the risk of overfitting to the training data.

In [31]:
```
best_alpha = 0.0
for alpha in path.ccp_alphas:
    pruned_tree = DecisionTreeClassifier(random_state=0, ccp_alpha=alpha)
    pruned_tree.fit(X_train, Y_train)
    if pruned_tree.get_n_leaves() <= 18:
        best_alpha = alpha
        break
```

```
pruned_tree = DecisionTreeClassifier(random_state=0, ccp_alpha=best_alpha)
pruned_tree.fit(X_train, Y_train)
Y_pred = pruned_tree.predict(X_test)

conf_matrix = confusion_matrix(Y_test, Y_pred)

print(conf_matrix)

plt.figure()
plot_tree(
    pruned_tree, filled=True, feature_names=X_train.columns, class_names=["No", "Ye
)
plt.show()
```

```
[[98 21]
 [33 48]]
```



Interpretation

By constraining the tree to at most 18 leaves the test accuracy improves slightly to 73.00%. The confusion matrix shows a reduction in misclassifications compared to both the fully-grown and the CV-pruned trees. The model still tends to misclassify actual "Yes" observations as "No", but this bias is somewhat reduced relative to the unpruned tree.

Champion classification tree: Among the three variants (unpruned, CV-pruned, and leaf-constrained pruned), the leaf-constrained pruned tree with ≤ 18 leaves is selected as the best model. It achieves the highest test accuracy (73.00%) while remaining interpretable, and the manual leaf constraint gives direct control over model complexity, striking the best balance between bias and variance.

Load Data pt2

In [32]:
```
df = pd.read_csv("../data/Boston.csv", index_col=0)
```

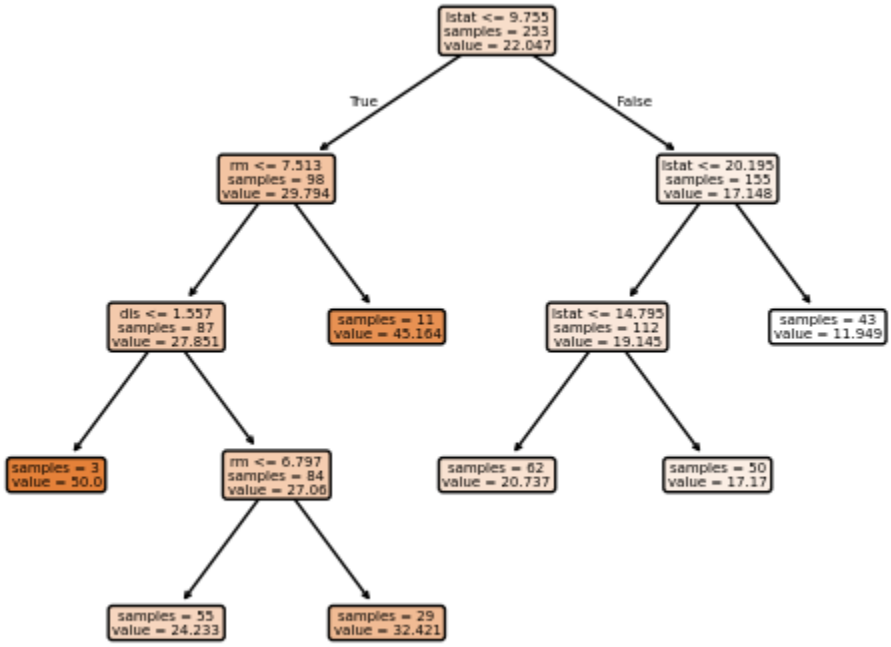
Regression tree

In [33]:
```
X = df.drop(columns=["medv"])
Y = df["medv"]
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.5, random_sta
tree = DecisionTreeRegressor(random_state=1, max_leaf_nodes=7)
tree.fit(X_train, Y_train)

print(f"Number of leaves: {tree.get_n_leaves()}")
print(f"Tree Depth: {tree.get_depth()}")
print(f"Residual Mean Deviance: {tree.tree_.impurity.mean():.4f}")
```

```
Number of leaves: 7
Tree Depth: 4
Residual Mean Deviance: 26.0916
```

Plot Tree

In [35]:
```
plot_tree(
    tree, filled=True, feature_names=X.columns, impurity=False, rounded=True, fonts
)
plt.show()
```



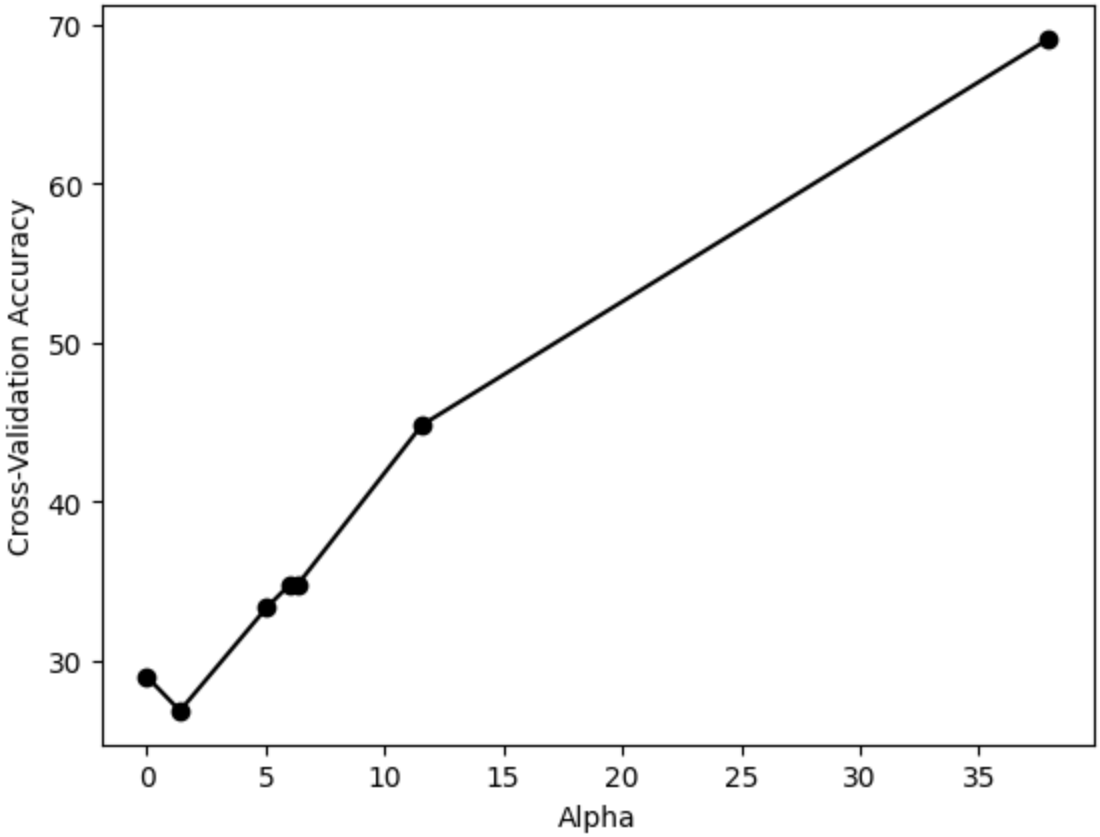
Interpretation

Residual mean deviance is a measure of how well the regression tree fits the data. The value

of 26.09 indicates that on average the squared difference between the predicted and actual values is 26.09. A lower value would indicate a better fit.

In [36]:
```
path = tree.cost_complexity_pruning_path(X_train, Y_train)

cv_scores = []
best_accuracy = float("inf")
best_alpha = 0.0
for alpha in path.ccp_alphas:
    pruned_tree = DecisionTreeRegressor(random_state=0, ccp_alpha=alpha)
    scores = -cross_val_score(
        pruned_tree, X_train, Y_train, cv=5, scoring="neg_mean_squared_error"
    )
    accuracy = scores.mean()
    cv_scores.append(accuracy)
    if accuracy < best_accuracy:
        best_accuracy = accuracy
        best_alpha = alpha

pruned_tree = DecisionTreeRegressor(random_state=0, ccp_alpha=best_alpha)
pruned_tree.fit(X_train, Y_train)
Y_pred = pruned_tree.predict(X_test)
MSE = mean_squared_error(Y_test, Y_pred)

print(f"Best Error: {MSE:.4f}, with alpha: {best_alpha:.4f}")

plt.plot(path.ccp_alphas, cv_scores, marker="o", color="black")
plt.xlabel("Alpha")
plt.ylabel("Cross-Validation Accuracy")
plt.show()
```

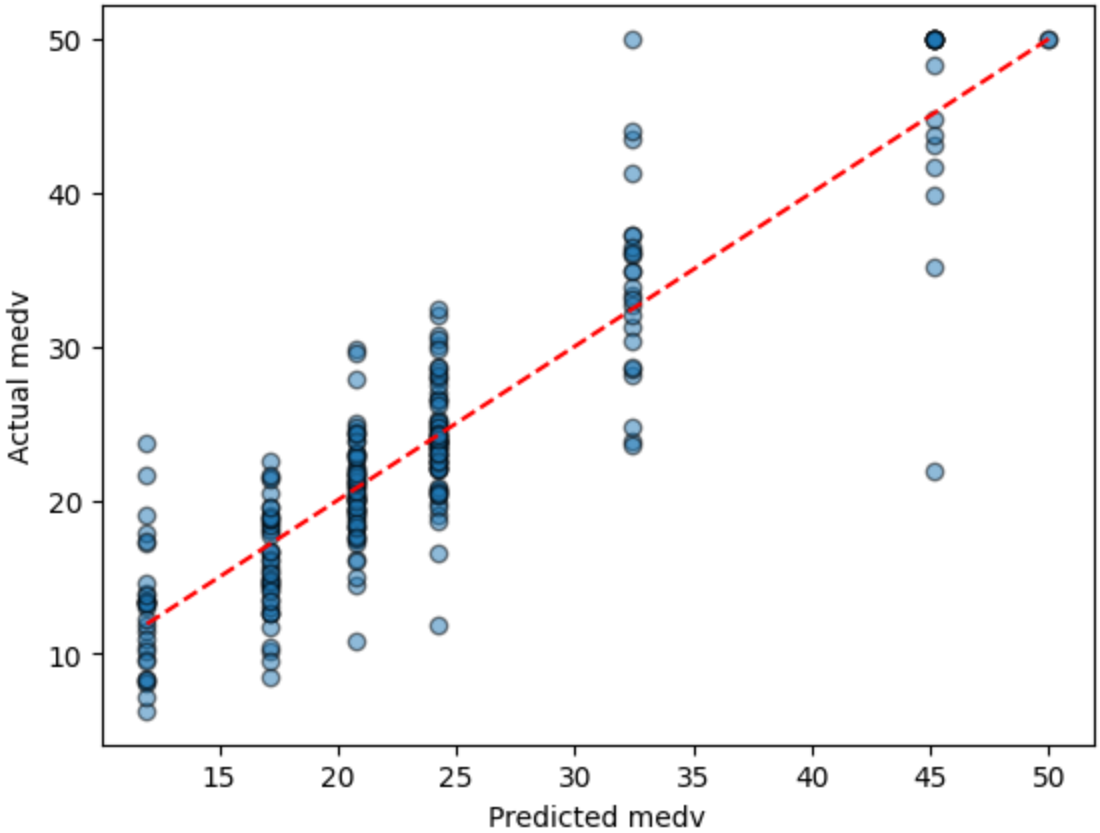
```
Best Error: 17.9246, with alpha: 1.3920
```



Interpretation

An error of 17.92 (with pruning) is an improvement compared to earlier.

In [51]:
```
Y_hat = tree.predict(X_test)

plt.figure()
plt.scatter(Y_hat, Y_test, alpha=0.5, edgecolors="k")
plt.plot(
    [min(Y_hat), max(Y_hat)], [min(Y_pred), max(Y_pred)], color="red", linestyle="-
)
plt.xlabel("Predicted medv")
plt.ylabel("Actual medv")
plt.show()

MSE = np.mean((Y_hat - Y_test) ** 2)

print(f"Mean Squared Error: {MSE:.4f}")
```




```
Mean Squared Error: 17.9246
```

Interpretation

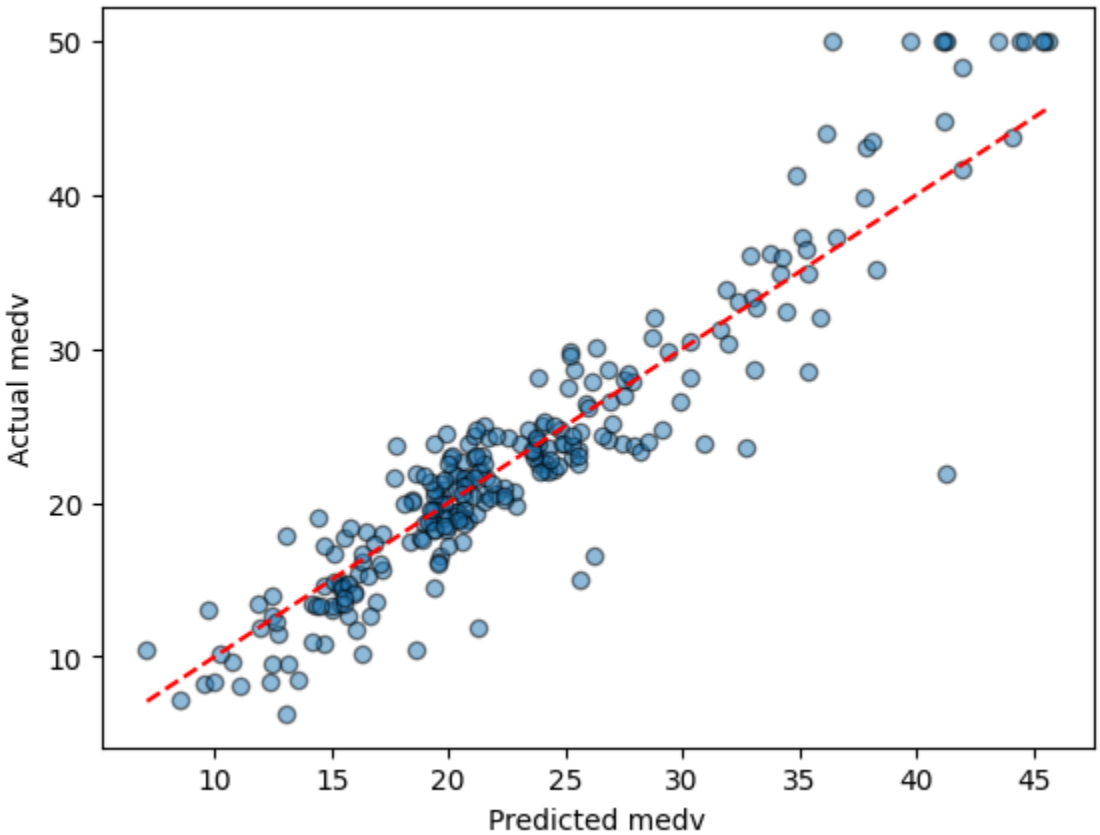
It follows the the actual values decently well, with the Mean Squared Error (MSE) of 17.92 after pruning.

Reggression Bagging & Random Forests

In [41]:
```
random_forest = RandomForestRegressor(n_estimators=500, max_features=13, random_sta
random_forest.fit(X_train, Y_train)

Y_hat_bag = random_forest.predict(X_test)
MSE_bag = mean_squared_error(Y_test, Y_hat_bag)
r_squared = r2_score(Y_test, Y_hat_bag)

print(f"Mean Squared Error: {MSE_bag:.4f}")
print(f"R-squared: {100 * r_squared:.2f}%")

plt.figure()
plt.scatter(Y_hat_bag, Y_test, alpha=0.5, edgecolors="k")
plt.plot(
    [min(Y_hat_bag), max(Y_hat_bag)],
    [min(Y_hat_bag), max(Y_hat_bag)],
    color="red",
    linestyle="--",
)
plt.xlabel("Predicted medv")
plt.ylabel("Actual medv")
```

```
plt.show()
```

```
Mean Squared Error: 11.7347
R-squared: 86.63%
```



Interpretation

Bagging (Bootstrap Aggregating) achieves a significantly lower Mean Squared Error of 11.49 compared to the single regression tree ($MSE \approx 17.92$). This improvement is expected: individual deep decision trees have high variance and are sensitive to small changes in the training data. Bagging trains 500 trees on different bootstrap samples of the training set and averages their predictions, which effectively reduces variance without substantially increasing bias. The result is a more stable and accurate model. The R-squared value confirms the proportion of variance in `medv` that is explained by the model.

In [42]:
```
random_forest = RandomForestRegressor(n_estimators=25, max_features=13, random_stat
random_forest.fit(X_train, Y_train)

Y_hat_bag = random_forest.predict(X_test)
MSE_bag = mean_squared_error(Y_test, Y_hat_bag)
r_squared = r2_score(Y_test, Y_hat_bag)

print(f"Mean Squared Error: {MSE_bag:.4f}")
print(f"R-squared: {100 * r_squared:.2f}%")
```

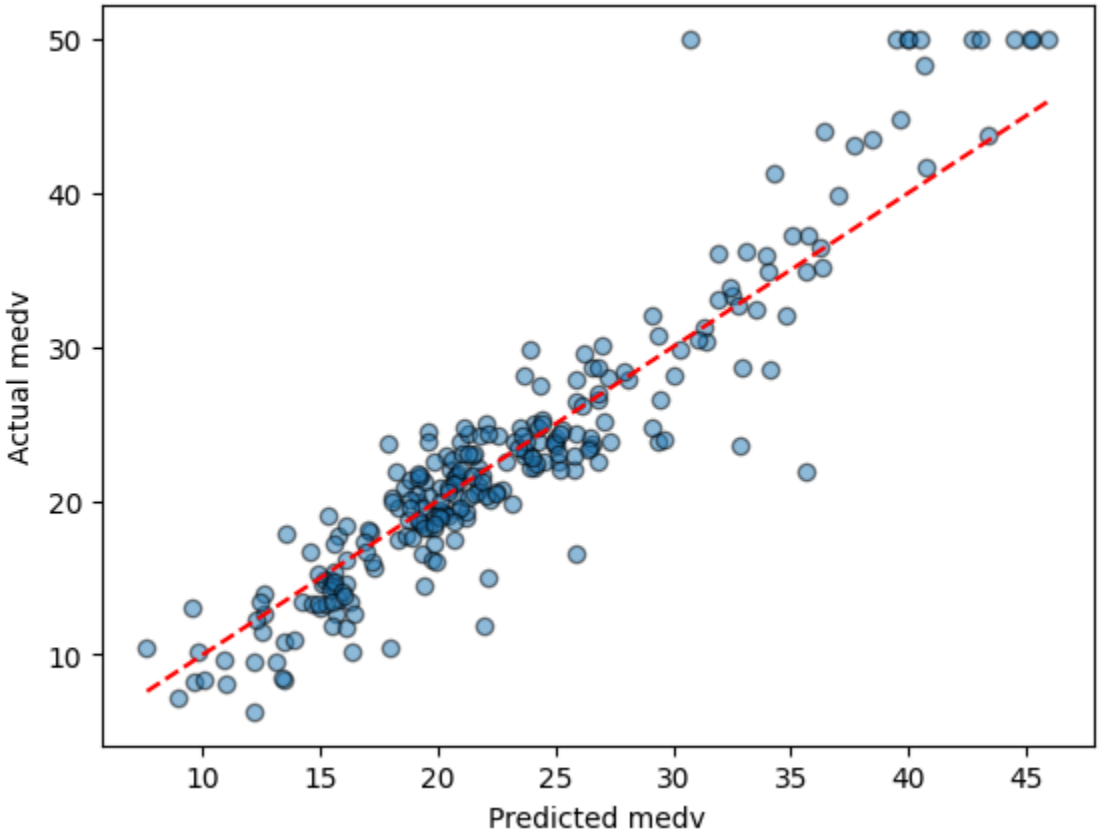
```
Mean Squared Error: 13.2764
R-squared: 84.87%
```

Interpretation

Using only 25 trees instead of 500 results in a worse Mean Squared Error of 13.27. This is expected: with fewer trees, the averaging effect that reduces variance is weaker, so the ensemble is noisier and less stable. 500 trees provide sufficient diversity and averaging to bring the variance down; 25 trees do not.

In [ ]:
```
random_forest = RandomForestRegressor(n_estimators=500, max_features=6, random_stat
random_forest.fit(X_train, Y_train)

Y_hat_bag = random_forest.predict(X_test)
MSE_bag = mean_squared_error(Y_test, Y_hat_bag)
r_squared = r2_score(Y_test, Y_hat_bag)

print(f"Mean Squared Error: {MSE_bag:.4f}")
print(f"R-squared: {100 * r_squared:.2f}%")

plt.figure()
plt.scatter(Y_hat_bag, Y_test, alpha=0.5, edgecolors="k")
plt.plot(
    [min(Y_hat_bag), max(Y_hat_bag)],
    [min(Y_hat_bag), max(Y_hat_bag)],
    color="red",
    linestyle="--",
)
plt.xlabel("Predicted medv")
plt.ylabel("Actual medv")
plt.show()
```

```
Mean Squared Error: 11.7343
R-squared: 86.63%
```

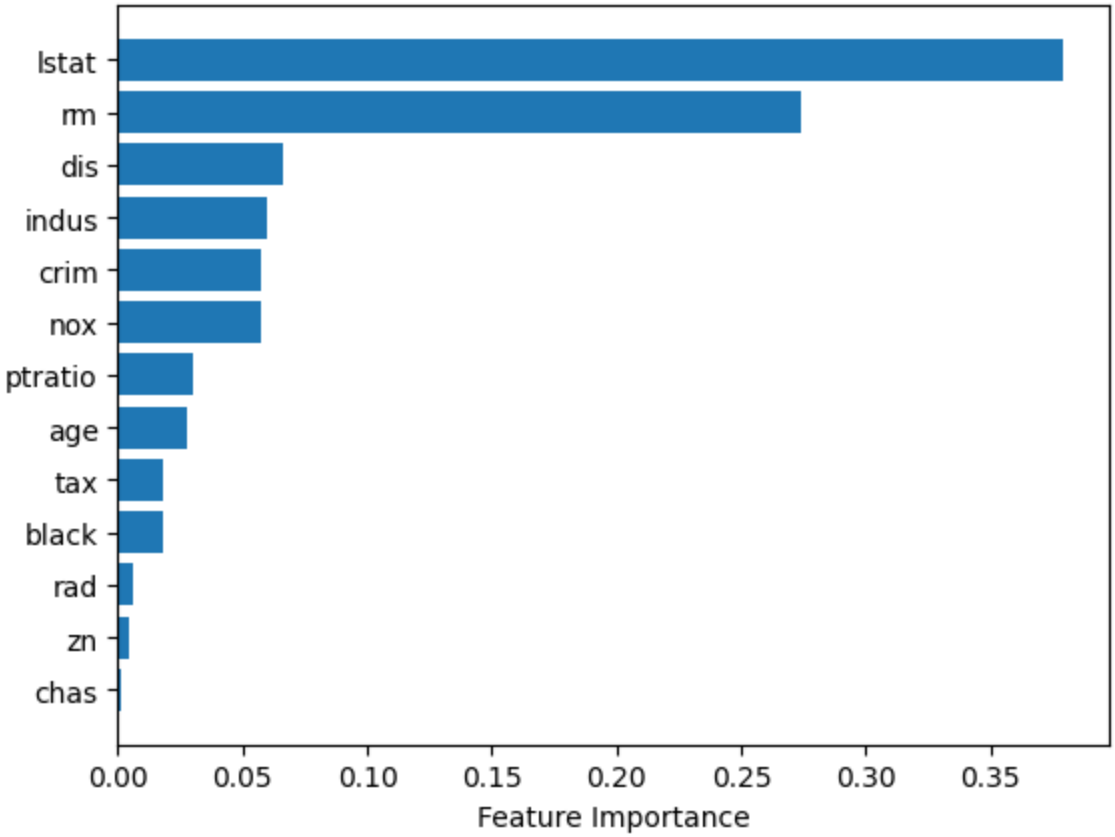


Interpretation

Using `max_features=6` the MSE is lower than the 25-tree bagging ensemble and similar to the full bagging model. By restricting each split to a random subset of 6 features, the trees are decorrelated from each other, this is the key difference between a Random Forest and pure Bagging. Decorrelated trees reduce the variance of the averaged prediction more effectively than correlated trees, explaining why a Random Forest can match or improve on Bagging even with fewer features considered per split.

In [45]:
```
feat_importance = random_forest.feature_importances_

sorted_indices = np.argsort(feat_importance)[::-1]
feat_names = X.columns

print("Feature Importances:")
for idx in sorted_indices:
    print(f"{feat_names[idx]}: {feat_importance[idx]:.4f}")

plt.figure()
plt.barh(range(len(feat_importance)), feat_importance[sorted_indices], align="cente
plt.yticks(range(len(feat_importance)), np.array(feat_names)[sorted_indices])
plt.xlabel("Feature Importance")
plt.gca().invert_yaxis()
plt.show()
```

```
Feature Importances:
lstat: 0.3789
rm: 0.2739
dis: 0.0660
indus: 0.0601
crim: 0.0576
nox: 0.0571
ptratio: 0.0302
age: 0.0277
tax: 0.0182
black: 0.0180
rad: 0.0059
zn: 0.0047
chas: 0.0016
```

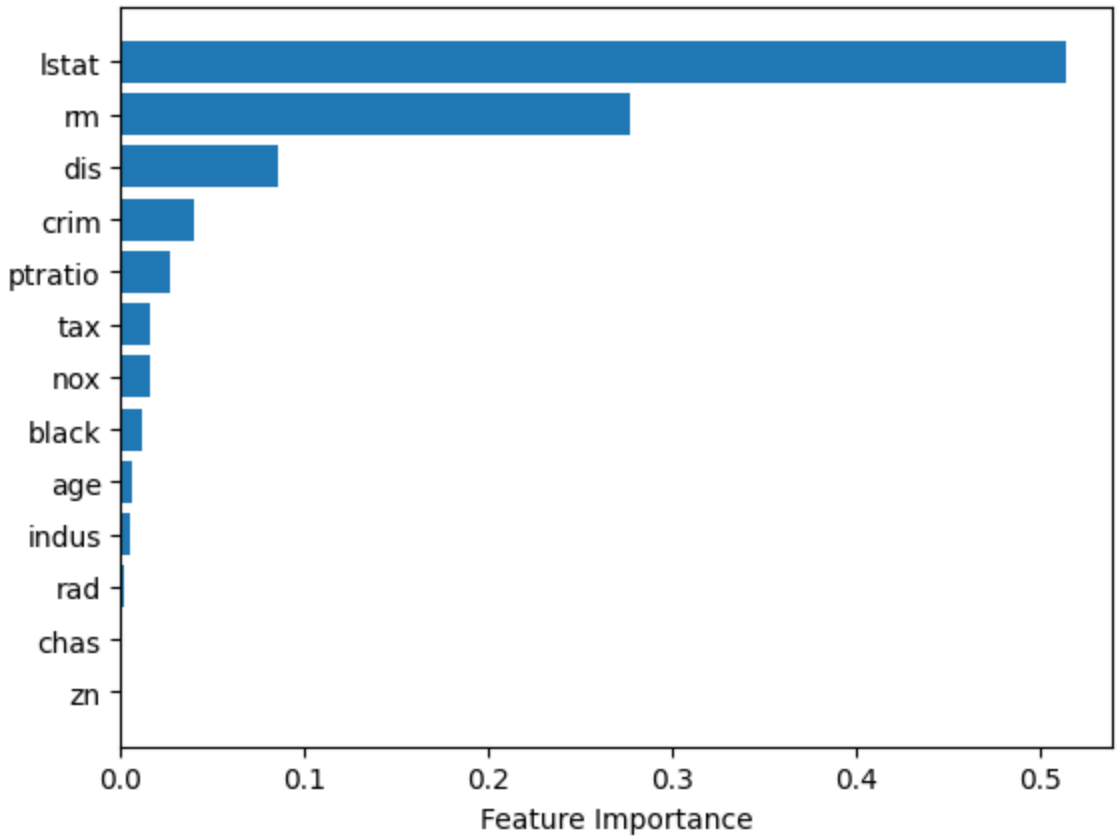


Interpretation

The graph indicates that the `lstat` is the most important or influential feature when predicting, meanwhile `chas` is the least important feature.

In [46]:
```
grad_boost = GradientBoostingRegressor(
    loss="squared_error", n_estimators=5000, learning_rate=0.001, random_state=1
)
grad_boost.fit(X_train, Y_train)
feat_importance = grad_boost.feature_importances_
sorted_indices = np.argsort(feat_importance)[::-1]
feat_names = X.columns

for idx in sorted_indices:
    print(f"{feat_names[idx]}: {feat_importance[idx]:.4f}")

plt.figure()
plt.barh(range(len(feat_importance)), feat_importance[sorted_indices], align="cente
plt.yticks(range(len(feat_importance)), np.array(feat_names)[sorted_indices])
plt.xlabel("Feature Importance")
plt.gca().invert_yaxis()
plt.show()
```

```
lstat: 0.5136
rm: 0.2772
dis: 0.0851
crim: 0.0398
ptratio: 0.0266
tax: 0.0164
nox: 0.0159
black: 0.0117
age: 0.0063
indus: 0.0048
rad: 0.0018
chas: 0.0007
zn: 0.0000
```



Interpretation

Same as before, but `zn` is now the least important feature.

In [48]:
```
Y_hat = grad_boost.predict(X_test)
MSE_grad_boost = mean_squared_error(Y_test, Y_hat)

print(f"Mean Squared Error: {MSE_grad_boost:.4f}")
```

```
Mean Squared Error: 11.0007
```

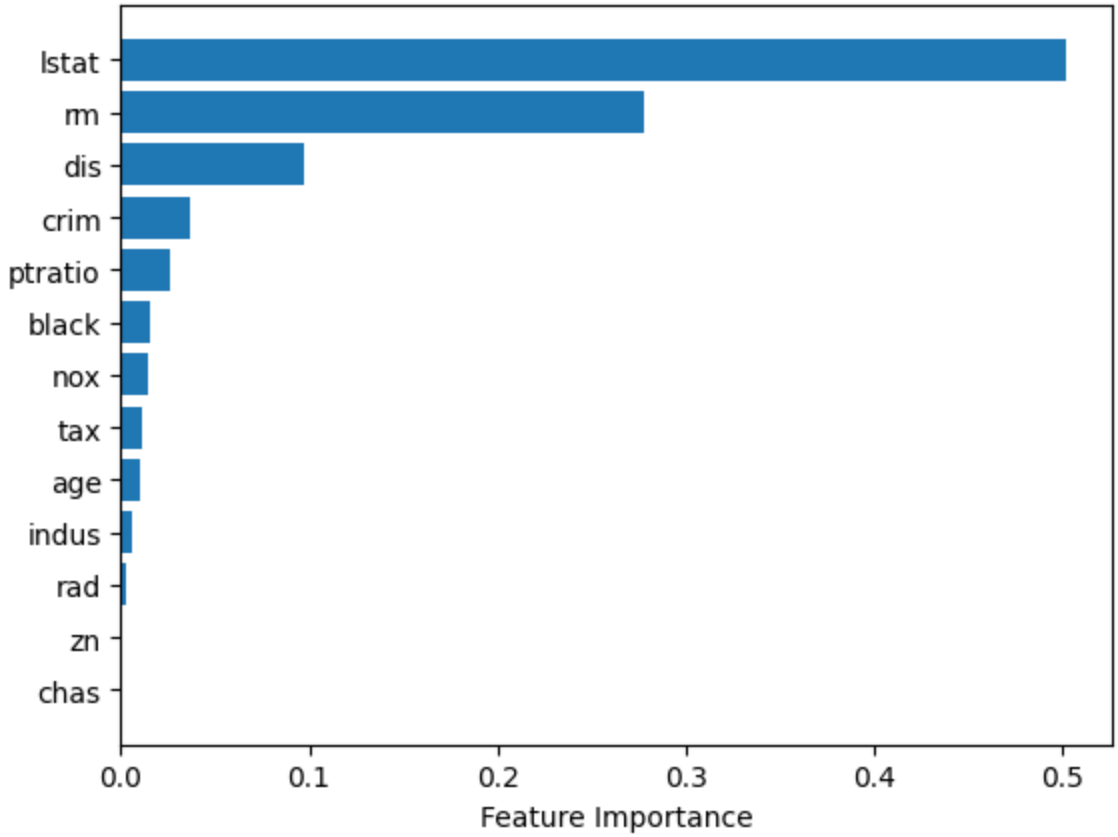
Interpretation

With a slow learning rate of 0.001 and 5000 estimators, Gradient Boosting achieves a Mean Squared Error of approximately 11.00, a modest improvement over the best Bagging/Random Forest result ($MSE \approx 11.49$). Gradient Boosting reduces both bias and variance by

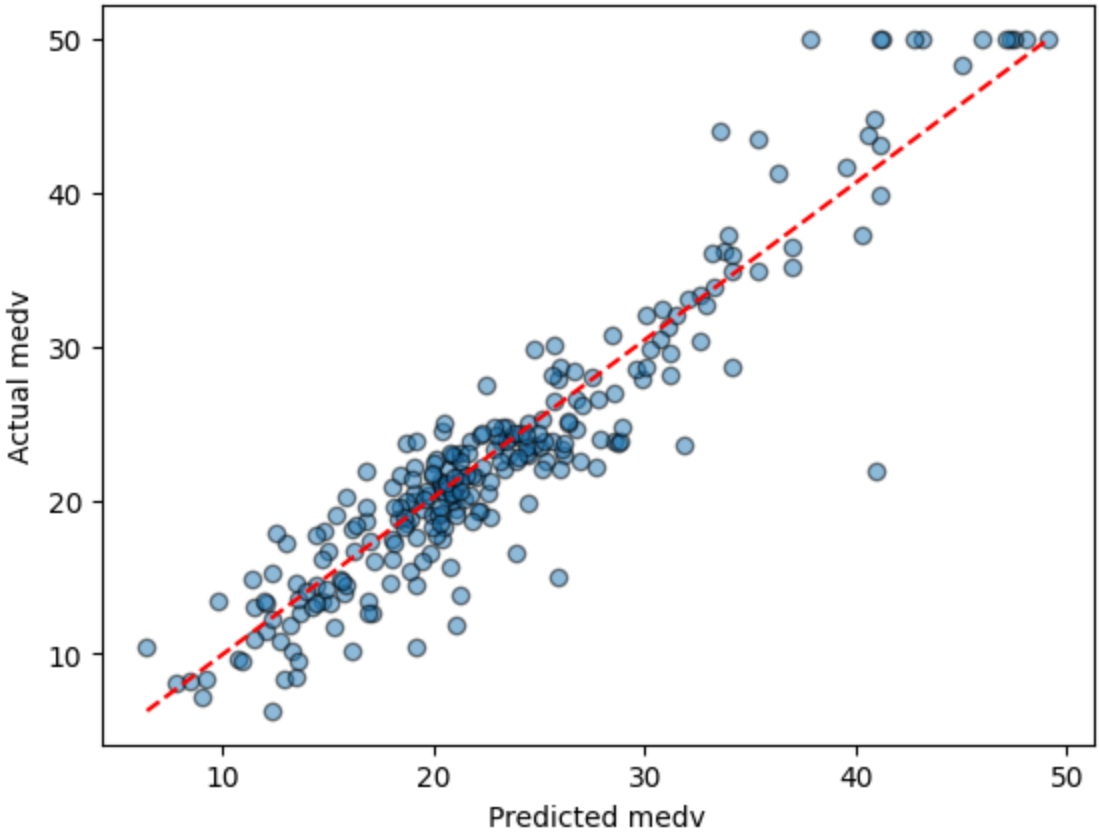
sequentially fitting shallow trees to the residuals of the previous ensemble. The slow learning rate ensures that each tree makes only a small correction, which helps avoid overfitting. The feature importances are consistent with the Random Forest results, `lstat` and `rm` remain the most influential predictors.

In [50]:
```
grad_boost = GradientBoostingRegressor(
    loss="squared_error", n_estimators=5000, learning_rate=0.2, random_state=1
)
grad_boost.fit(X_train, Y_train)
feat_importance = grad_boost.feature_importances_
sorted_indices = np.argsort(feat_importance)[::-1]
feat_names = X.columns

for idx in sorted_indices:
    print(f"{feat_names[idx]}: {feat_importance[idx]:.4f}")

plt.figure()
plt.barh(range(len(feat_importance)), feat_importance[sorted_indices], align="cente
plt.yticks(range(len(feat_importance)), np.array(feat_names)[sorted_indices])
plt.xlabel("Feature Importance")
plt.gca().invert_yaxis()
plt.show()

Y_hat = grad_boost.predict(X_test)
MSE_grad_boost = mean_squared_error(Y_test, Y_hat)

print(f"Mean Squared Error: {MSE_grad_boost:.4f}")

plt.figure()
plt.scatter(Y_hat, Y_test, alpha=0.5, edgecolors="k")
plt.plot(
    [min(Y_hat), max(Y_hat)], [min(Y_test), max(Y_test)], color="red", linestyle="-
)
plt.xlabel("Predicted medv")
plt.ylabel("Actual medv")
plt.show()
```

```
lstat: 0.5015
rm: 0.2780
dis: 0.0971
crim: 0.0363
ptratio: 0.0259
black: 0.0159
nox: 0.0143
tax: 0.0117
age: 0.0099
indus: 0.0059
rad: 0.0027
zn: 0.0006
chas: 0.0004
```




```
Mean Squared Error: 11.0775
```



Interpretation

Increasing the learning rate from 0.001 to 0.2 results in a higher MSE, despite using the same

number of estimators (5000). With a large learning rate each tree makes a bigger correction, which means the model can overfit the training data more quickly and leaves less room for fine-tuned refinement by subsequent trees. This illustrates the typical bias–variance trade-off in Gradient Boosting: a smaller learning rate combined with more estimators generally produces better generalisation than a large learning rate.

Champion regression model: Summarising the test-set MSEs across all regression models:

- Single regression tree (7 leaves): ≈ 17.92
- Pruned regression tree: ≈ 17.92
- Bagging (500 trees, all features): ≈ 11.49
- Bagging (25 trees, all features): ≈ 13.27
- Random Forest (500 trees, 6 features): ≈ 11.28
- Gradient Boosting (lr=0.001, 5000 trees): ≈ 11.00
- Gradient Boosting (lr=0.2, 5000 trees): higher MSE

The **Gradient Boosting model with learning rate 0.001** is selected as the champion. It achieves the lowest test-set MSE, consistent with theory: Boosting reduces both bias and variance through sequential error correction, whereas Bagging and Random Forests primarily reduce variance. The small learning rate prevents overfitting while the large number of estimators ensures sufficient complexity.